

File Systems

Files and Directories

Metadata Data

(Who owns the file,
filename,

when was the file created,
file accessed

data block addresses.

attributes

Hierarchical structure to logically group
the data stored on secondary storage.

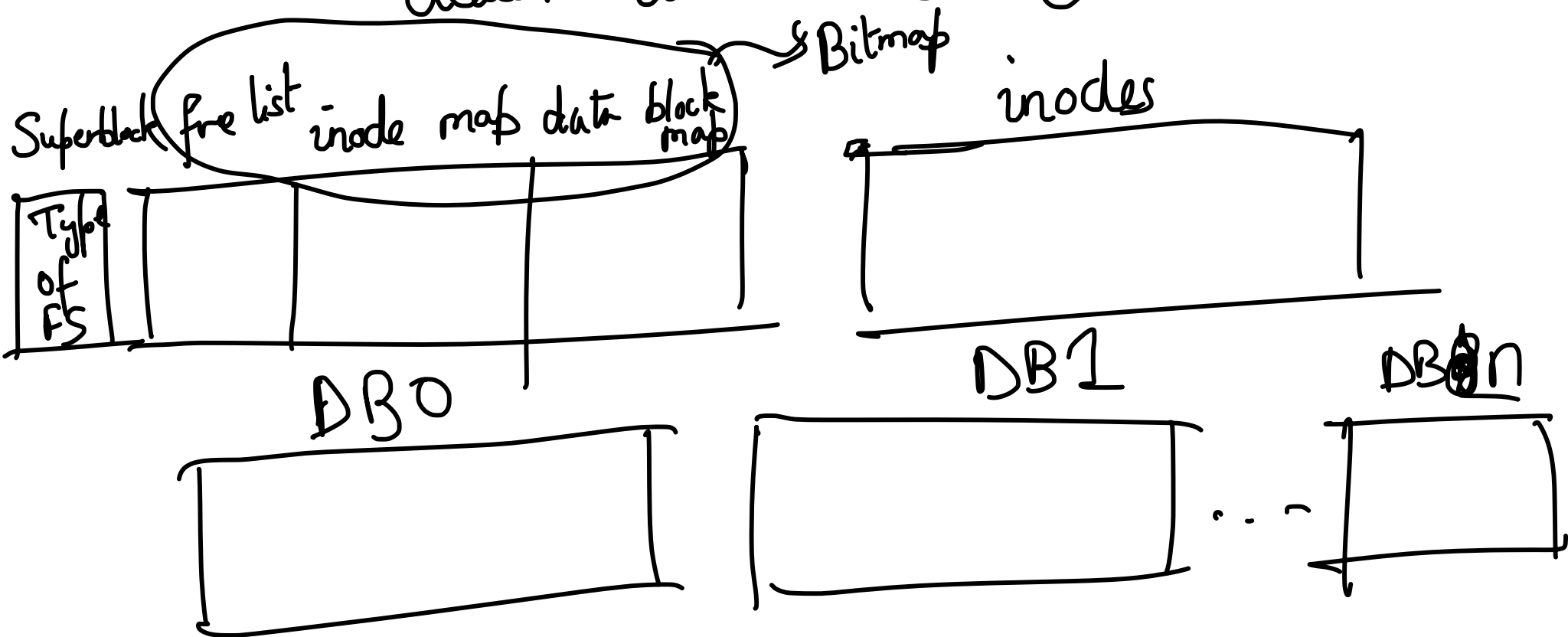
Filesystem used for each secondary
storage device → Part of the kernel

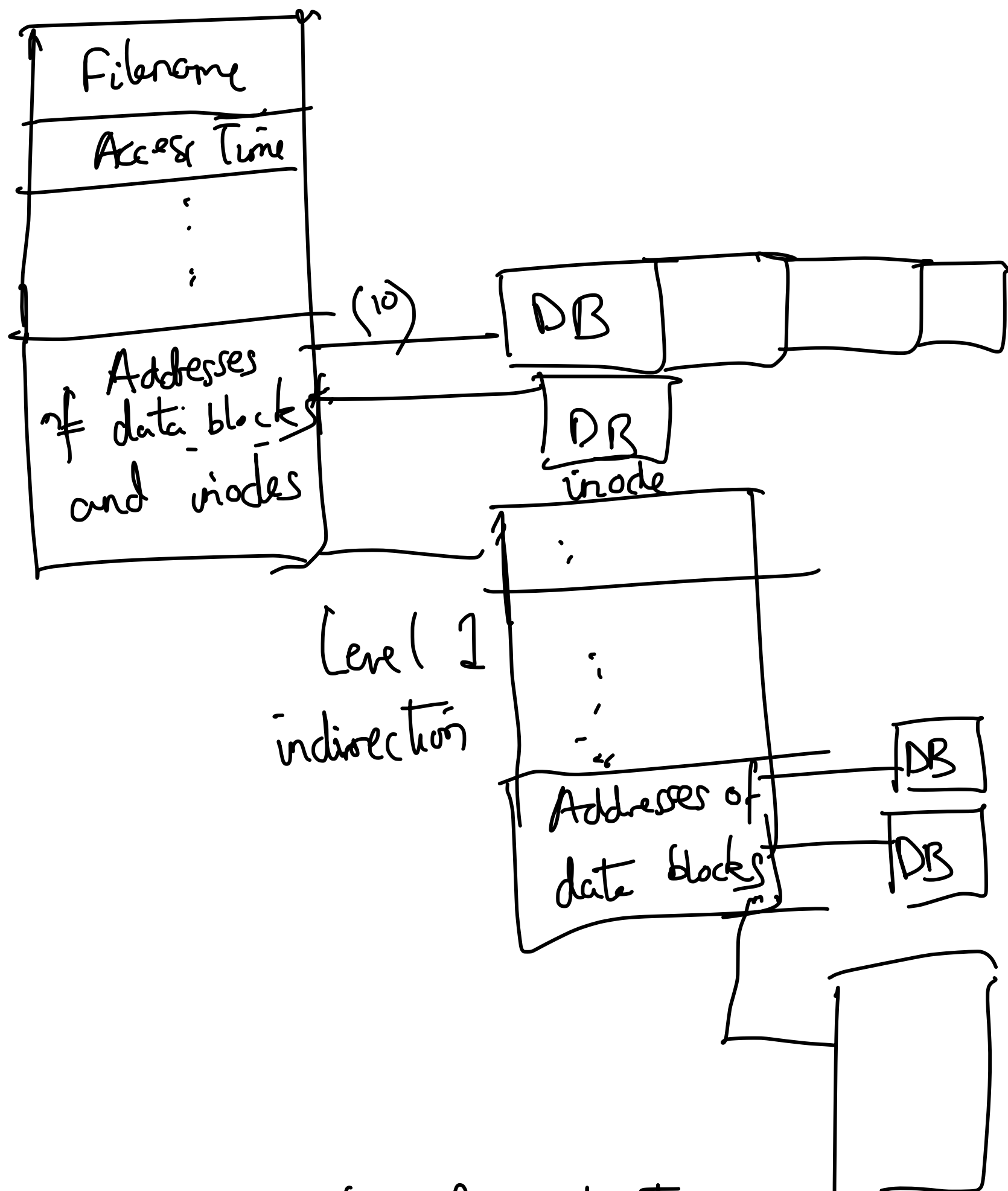
and decides how the data should be stored -

ext4 $\rightarrow$ most common file system in

ext2 $\rightarrow$ 1) The ones used for storing metadata are called inodes

2) The ones used for storing actual data are called data blocks.





The more the level of indirection, you can store more information in a single file.

The disadvantage is that it leads to addition

of Time complexity.

What is the problem with the above basic file system?

(1) Does not take into account the principle of locality $\rightarrow$ ^(a) Have a more intelligent data block and inode allocation scheme.

(b) Use of extants $\rightarrow$ A series of contiguous data blocks allocated together.

File System Calls

1) open () $\rightarrow$

abc.txt
/home/user/abc.txt

2) read ()

3) write ()

4) close ()

1) $\swarrow^{\textcircled{1}}$ /home / $\swarrow^{\textcircled{2}}$ user / $\swarrow^{\textcircled{3}}$ abc.txt $\swarrow^{\textcircled{4}}$

Directories are special files that contain a list of files.

2) read() $\longrightarrow$ ① Go to the file's inode

② Go to the relevant data block.

③ Write to the inode to change the file's last accessed attribute.

3) write() $\longrightarrow$ Apart from the blocks accessed by read, might also require bit map access to allocate data blocks and inodes.

4) close() $\longrightarrow$ Modify the actual inode to update the metadata.

Optimization Technique of ext4 : $\longrightarrow$ Keep the

directories and sub-directories in adjacent data blocks.

Keeps groups of free blocks together, and for each directory created, assigns a group (not a single data block).

Losing power or crash of a system during a write will leave the filesystem in an inconsistent state -

→ Do a scan of the entire filesystem and check whether the data structures (bitmap, inodes, superblock) are all consistent. (Can ~~lose~~ take hours on large filesystems). fsck command

→ Concept of Journals got introduced

→ Add another special disk block called journal.

→ Before every write, store the inode / data block number where it is about to happen (must be done in sync mode).

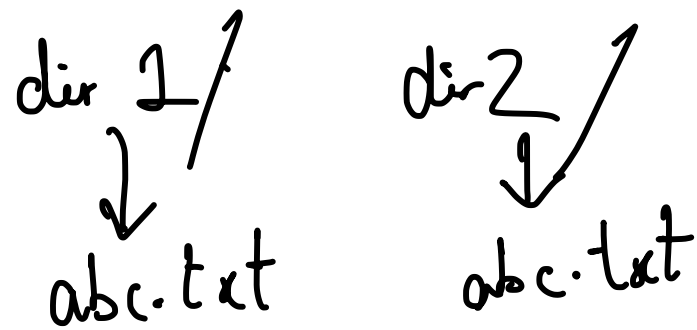
→ Once the write is completed, again store on the journal that it is complete.

Ext4 and NTFS use journals.

Tree structure is violated only in a few cases.

1) Soft Link → Similar to a shortcut, where the inode simply stores the address of another inode.

2) Hard link →



So inodes allow the flexibility to deviate from the hierarchical structure,